Lecture 10/27/26

Regular v.s. Nonregular

Regular = simple rules to check on the fly w/ fixed memory.

Nonregular = complex rules that require unlimited counting / memory

Non-Deterministic Finite Automata

$\delta(q_0, a) = \{q_1\}$ = Deterministic

In nondeterministic, this would be a set.

M1 = M2 if they accept the same language (same set of strings)

L(M) = Language of Machine

Non-Regular Languages - infinite memory

eg $L = \{a^n b^n : n \geq 0\}$, there is no DFA/NFA in existence that can accept it bc it requires unlimited counting memory.

$M = (Q, \Sigma, \delta, q_0, F)$

Q: set of states
Σ: Input alphabet ie. $\{a, b\}$
δ = Transition function
q_0 = Initial State

Regular Expressions - describe regular languages

Eg. $(a+b \cdot c)^*$

Recursive Definition

$\emptyset, \lambda, \propto$ (primitive)
r_1 & r_2 = Regular Expression

$r_1 + r_2$
$r_1 \cdot r_2$
r_1^*
(r_1)

] Regular Expressions

$(a+b \cdot c)^* \cdot (c + \emptyset)$ ← Empty set

Not regular - $(a+b+)$

$L(r)$ = Language of regular expression

$L(\emptyset) = \{\emptyset\}$ $\quad L(\propto) = \{\propto\}$

$L(\lambda) = \{\lambda\}$

+ Means Union

Union

$L(r_1 + r_2) = L(r_1) \cup L(r_2)$

$L(r_1 \cdot r_2) = L(r_1) L(r_2)$ (concatenation)

$L(r_1^*) = (L(r_1))^*$

$r = (aa)^* (bb)^* b$

$L(r) = \{a^{2n} b^{2m} b : n, m \geq 0\}$

* indicates previous pattern can be repeated 0, 1, 2 or an arbitrary # of times.

$pq = p \cdot q = 0^* 1 = \{1, 01, 001, \ldots\}$

↑ Concatenation

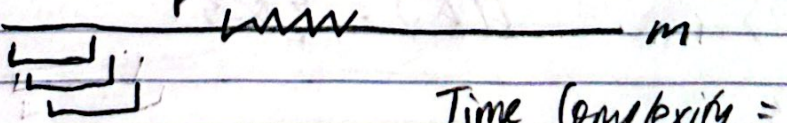
Union (OR) $p+q$ or $p \cup q$ = Either p or q Eg. $(0+1) = \{0, 1\}$

Fundamental Pattern Matching Problem

pattern $P = p_1 p_2 \cdots p_e$ $p_i \in \Sigma$

text $t = t_1, t_e, \ldots, t_m$ $t_j \in \Sigma$

Naive Approach - Sliding Window



Time Complexity = $O(m * L)$

BAD!

Finding occurrence of pattern in string !*

Knuth-Morris Pratt Algorithm (KMP-Algorithm)

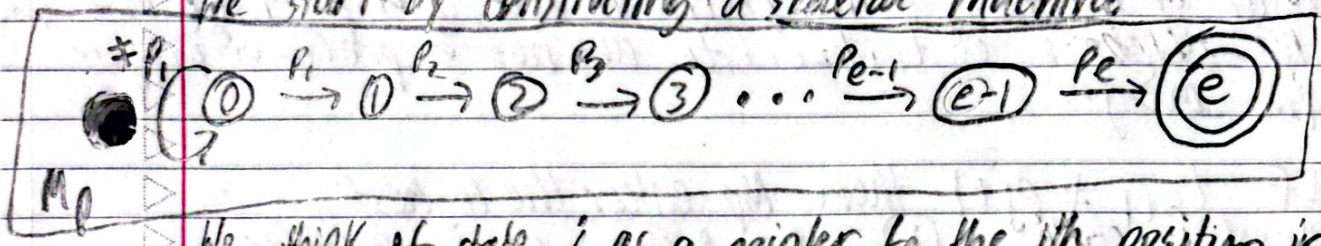
Time complexity - Linear - $O(m+l)$

$p = p_1 p_2 \cdots p_k$

$t = t_1, t_2 \cdots, t_n$

We first construct a pattern matching machine M_p.

We start by constructing a skeletal DFA machine



We think of state i as a pointer to the ith position in pattern p. M_p is like a DFA except some part of it.

We will extend the skeletal machine to a DFA M_p.

The state j of M_p corresponds to the prefix $p_1 p_2 \cdots p_j$

M_p starts in state 0 with its reading head at the first symbol on the input tape t_1

text $t_1, t_2, \cdots, t_m$.

If $t_1 = p_1$ then Mp enters state (1) and advances its reading head to position 2 on the input tape namely to t_2.

If $t_1 \neq p_1$ then Mp remains in state (0) and advances its input reading reading head to position 2 on the text/input tape. ie. t_2

Suppose after having "read" $t_1 t_2 \ldots t_k$ we have Mp is in state (j) (generic state). This implies that the last j symbols of $t_1 t_2 \ldots t_k$ are $p_1 p_2 \ldots p_j$. suffix of length j is the prefix of size j of p & the last m symbols of $t_1 t_2 \ldots t_k$ are not a prefix of $p_1 p_2 \ldots$ for $m > j$.

If $t_{k+1} \neq p_{j+1}$ then Mp enters the highest number state (i) s.t. $p_1 p_2 \ldots p_i$ is a suffix of $t_1 t_2 \ldots t_{k+1}$ to determine (i) Mp uses a failure function algorithm: f is the failure function: $f(j)$ is the largest s less than j for which $p_1 p_2 \ldots p_s$ is a suffix of $p_1 p_2 \ldots p_j$.

Scanned with CamScanner